PREDICTING BUS RIDERSHIP IN TRONDHEIM: A MACHINE LEARNING APPROACH TO PUBLIC TRANSPORTATION DEMAND FORECASTING Øyvind Sarheim Table of Contents Section Title Page

1. Introduction 3
2. Background 4
3. Problem statement 8
4. Project objectives 9
5. Data warehouse details 10
6. Interesting insights 13
7. Data pre-processing 19
8. Measuring performance 22
9. Algorithms applied 24 9.1 Random Forest Algorithm 25 9.2 XGBoost Algorithm 28 9.3 Neural Network Algorithm 31 9.4 Model Comparison 34
10. Discussion 36
11. Conclusion 40 References 42
12. Introduction Public transportation systems serve as the backbone of urban mobility, facilitating daily movement for commuters, students, event-goers and diverse city residents. For public transport operators like AtB in Trondheim, managing this transport system presents a complex operational challenge that requires strategic resource allocation, optimal route planning and efficient bus deployment. Accurate passenger volume forecasting emerges as a critical capability that directly impacts operational efficiency and service quality. Research Objectives and Methodology This project aims to investigate the application of machine learning techniques for bus ridership forecasting. By leveraging a comprehensive dataset from the whole of 2024, that integrates passenger counts, meteorological data and local event information, three distinct machine learning models were developed and systematically evaluated to assess whether data-driven approaches can generate sufficiently robust forecasts that support AtB's operational decision-making processes. Key Findings and Contributions The research yielded promising results, with the optimized Random Forest model achieving a Mean Absolute Error (MAE) of 1.38 passengers per stop-hour prediction. This performance represents a 13.75% improvement over baseline methods (MAE of 1.60), accomplished through advanced feature engineering techniques and systematic parameter optimization.

Key Performance Metrics (All values to 2 decimal places): • Best Model MAE: 1.38 passengers • Baseline Random Forest MAE: 1.60 passengers • Improvement: 13.75% • R^2: 0.51

Report Structure This report documents the complete process, encompassing data processing, model development and comparative evaluation methodologies. The findings provide transparent insights into both the potential applications and inherent limitations of machine learning approaches in public transportation demand forecasting. 2. Background The mobility and movement of people in a city constitute a cornerstone of modern urban functionality, directly impacting economic productivity, social equity and environmental sustainability (Smith., et al. (2020). Public transportation systems play an essential role in contributing to this mobility. This section provides an overview of public transport operations and ridership analysis, highlighting the complexities and challenges that create opportunities for machine learning applications.

2.1 The Role of Public Transportation in Urban Environments Public transportation serves as a vital part of the urban landscape, offering sustainable alternatives to private vehicle use. The benefits are multifaceted: reduced traffic congestion, lower greenhouse emissions, and improved air quality. (Movaghar, a.,et al., 2025). Beyond environmental considerations, public transportation promotes social inclusion by providing mobility options for populations without access to private cars, including students, elderly residents, and lower-income demographics (Göransson, J., & Andersson, H. (2023). In Trondheim, with its distinctive topographical features and significant student population, an effective public transportation system is essential for daily operations and overall urban efficiency. Trondheim’s unique characteristics, including seasonal population fluctuations due to academic calendars and challenging winter weather conditions, create specific operational requirements for public transport providers.

2.2 Operational Challenges in Public Bus Services Managing a public bus network presents multiple operational complexities, with passenger demand variability representing the most significant challenge. Transit operators like AtB face a constant task of balancing service provisions with fluctuating ridership patterns to ensure both passenger satisfaction and operational efficiency.
2.2.1 Supply-Demand Matching The challenge lies in matching service supply with dynamic passenger demand. Over-provisioning leads to underutilized vehicles, increasing operational costs through unnecessary fuel consumption, maintenance expenses, and driver wages while contributing to negative environmental impact. Conversely, under-provisioning leads to overcrowded buses, passenger discomfort, and service reliability issues when buses cannot accommodate waiting passengers. This supply-demand balancing represents a fundamental optimization problem in public transportation operations . Modern approaches integrate intelligent transportation systems with data-driven optimization techniques, achieving significant operational cost improvements (Fatorachian., et al., 2025). 2.2.2 Resource Allocation Efficiently allocating resources, including buses and drivers, across the different routes and different times of day is critical. Decisions regarding route scheduling, fleet deployment, and shift allocation must be made based on anticipated passenger loads (Ceder, 2016). The complexity increases when considering bus capacity constraints, driver availability and maintenance scheduling requirements. 2.2.3 Cost Management Public transport operations are often

under tight budgetary constraints. Optimizing resource utilization to minimize unnecessary expenditure while maintaining a high level of service represents a continuous operational objective (Kepaptsoglou., et al., 2009).

2.3 Factors influencing public transport ridership Public transport demand exhibits complex, multi-dimensional patterns influenced by various temporal, environmental, and social factors. Understanding these drivers is essential for developing accurate predictive models. 2.3.1 Temporal patterns Ridership demonstrates distinct patterns throughout the day, week, month and year. As can be seen in Figure 1: Daily patterns typically feature pronounced morning and evening peaks, corresponding to commuter travel, with reduced midday demand, as illustrated in Figure 1. Weekly patterns show variations between weekdays and weekends, while seasonal fluctuations are particularly pronounced in university cities like Trondheim, where academic calendars significantly impact ridership volumes. Recent research has emphasized the importance of mining temporal behaviour patterns for transport prediction applications (Foell, S., et al,.2016). These studies demonstrate that historical usage patterns contain predictive signals that can inform future demand forecasting, particularly when combined with contextual information about events and environmental conditions.

Figure 2.1: A typical daily ridership curve for a metropolitan transit system showing distinct morning and evening peaks, with reduced midday demand (Luo., et al,. 2019).

2.3.2 Weather and Environmental Factors Meteorological conditions play a significant role in public transport usage, with precipitation, temperature and seasonal changes affecting passenger behaviour and route preferences. In Nordic cities like Trondheim, winter weather conditions create particularly complex ridership patterns that must be incorporated into predictive models.

2.3.3 Local Events and Context Local events, university schedules, and cultural activities create demand variations that require sophisticated modelling approaches to capture accurately. Understanding these contextual factors is crucial for developing robust predictive systems that can adopt to local operational environments (Cats, O., et al,. 2021).

2.3.4 Service Quality Service quality parameters significantly influence passenger choice and ridership patterns. Critical factors include service frequency, punctuality, travel time, comfort, and safety, all of which serve as primary drivers of ridership demand (Dell'Olio., et al,. 2011). The interdependent relationship between service quality and ridership creates a feedback loop where improved service attracts more passengers, while declining quality can lead to a decrease in passengers and reduced system efficiency.

2.4 The Strategic Importance of Ridership Forecasting Given the operational complexities outlined above, accurate ridership forecasting represents a critical capability for public transport operators. Reliable forecasting systems enable multiple operational improvements across different time horizons and decision-making levels. Operational Benefits include proactive service adjustments, such as deploying additional vehicles anticipating peak demand periods caused by major events or adverse weather conditions. Resource allocation optimization allows for strategic bus deployment and staff scheduling aligned with predicted demand patterns, resulting in significant cost reductions while maintaining service quality. Passenger Experience Enhancement occurs through minimized overcrowding and reduced waiting times via improved supply-demand matching. Data-driven insights support strategic planning initiatives, including route network design, infrastructure investment decisions, and fare policy development. Historically, public transport forecasting has relied primarily on historical averages and manual adjustments based on operator experience. However, the increasing availability of granular operational data—including automated fare collection systems, real-time vehicle location data, and mobile ticketing platforms—combined with advances in computational techniques, has created new opportunities for sophisticated, data-driven forecasting approaches (Hounsell., et al., 2012). AtB has actively embraced these technological developments, implementing advanced data collection systems that provide the foundation for machine learning applications (AtB Annual Report, 2024).

2.5 Machine Learning Applications and Predictive Analytics Machine learning (ML) represents a subfield of artificial intelligence that provides tools and techniques for computers to learn patterns from data without being explicitly programmed for each specific task. In ridership forecasting contexts, ML algorithms analyze historical datasets-encompassing ridership patterns, meteorological conditions, event schedules and other contextual factors-to construct predictive models capable of generating accurate future demand estimates. This project investigates the application of three distinct ML algorithms to evaluate their effectiveness in generating reliable forecasts for Trondheim's bus network, thereby contributing to the broader objective of data-informed public transport optimization.

3. Problem statement The challenge of generating consistently reliable and granular short-to-medium-term bus ridership forecasts in Trondheim significantly impedes AtB's capacity to optimally align service supply with fluctuating passenger demand across different routes, time periods, and varying external conditions. Public transportation ridership exhibits complex dynamics influenced by temporal factors, meteorological conditions and local events, creating substantial operational challenges for transit operators. While qualitative understanding of demand drivers exists, quantitatively predicting future ridership volumes at granular levels—specific routes, stops, or time intervals—with sufficient accuracy to enable proactive service optimization remains a significant operational challenge. This challenge is well-documented in contemporary transportation research, where accurate short-term ridership forecasting has been identified as crucial for operational efficiency and service quality enhancement.

Research Gap: The specific challenge lies in accurately predicting passenger boarding numbers at individual bus stops for specific routes within defined time periods, accounting for the complex interplay of temporal, environmental, and contextual variables that influence ridership behaviour in Trondheim's unique operational environment. Practical Implications: Without reliable granular forecasting capabilities, AtB faces ongoing difficulties in resource allocation, service planning, and passenger experience optimization, potentially resulting in operational inefficiencies and suboptimal service delivery.

4. Project Objectives The primary objective of this research is to develop and evaluate machine learning models for predicting short-to-medium-term bus ridership in Trondheim's public transportation network. To achieve this overarching goal, seven specific sub-objectives were established:

4.1 Data Infrastructure Development This objective encompassed the identification, acquisition, cleaning, integration, and preparation of comprehensive datasets for machine learning analysis. The process involved merging three primary data sources: historical ridership data from AtB's operational systems, meteorological data from Norway's national weather service (frost.met.no), and contextual event data compiled from multiple sources including local government records. The integration challenge required reconciling datasets with different temporal granularities and formats while ensuring data quality and completeness across an 18.7 million record dataset spanning the entire 2024 calendar year. This foundation was critical for enabling sophisticated analytical approaches and ensuring model reliability.

4.2 Exploratory Data Analysis and Pattern Recognition Comprehensive exploratory data analysis (EDA) was conducted to uncover underlying patterns, trends, and relationships within the integrated dataset. The analysis focused on identifying key temporal patterns (daily, weekly, and seasonal ridership cycles), quantifying weather impacts on passenger behavior, and assessing the influence of special events on ridership demand.

4.3 Machine Learning Model Development Three distinct machine learning algorithms were developed and implemented to predict bus ridership at individual stop and route levels. The models were designed to handle complex, non-linear relationships between environmental factors, temporal patterns, and ridership behavior. Comparative performance evaluation was conducted to identify the most effective modeling approach for this specific domain, with particular attention to model robustness and generalizability across different operational conditions.

4.4 Model Evaluation and performance Optimization Robust evaluation frameworks were established to assess model performance using both standard machine learning metrics (MAE, R-squared) and domain-specific criteria reflecting operational realities of public transport planning. This dual approach emphasized the importance of model interpretability and practical applicability alongside predictive accuracy. Business-relevant evaluation criteria were developed to ensure that technical performance translated into operational value for transport operators.

4.5 Advanced Feature Engineering and Temporal Modeling Sophisticated feature engineering techniques were developed to enhance model accuracy in capturing complex dependencies and patterns inherent in ridership data. This included creating lag features to model historical ridership influence, rolling window statistics to capture short-term trends, and interaction features to model combined effects of weather and events. This objective revealed that raw data alone was insufficient for accurate prediction, highlighting the crucial role of domain-specific feature creation in achieving optimal model performance.

4.6 Business Impact Analysis and Operational Integration Technical model performance was translated into quantifiable business value for AtB and the broader Trondheim public transport ecosystem. Analysis focused on how improved ridership prediction could enhance operational efficiency, reduce costs, improve customer satisfaction, and support strategic planning decisions. This objective emphasized the practical implementation aspects of developed models within existing operational systems, ensuring research outcomes could be effectively deployed in real-world environments.

5. Data Warehouse Details The foundation of this project consists of a comprehensive data integration pipeline that combines three distinct data sources into a unified analytical dataset. This section details the architecture, acquisition processes, and integration methods that enabled the creation of a machine learning framework capable of modeling the complex relationships between various data attributes.

5.1 Data Sources Overview The integrated dataset incorporates information from three primary data sources, each contributing unique dimensions to the analytical framework: Data Source Provider Coverage Records Key Variables Bus Ridership Data AtB (Trondheim Transit Provider) 2024(12 monthly files) 18.7M records Route ID, Stop sequences, Boarding/alighting counts, Temporal data Meteorological Data Norwegian Meteorological Institute (frost.met.no) 2024 (hourly) 8.760 records Temperature, precipitation, wind speed, humidity Event Data Multiple sources (manual compilation) 2024 8.748 records RBK games, holidays, cultural events, academic calendar

5.2 Data Acquisition and Processing

5.2.1 AtB Ridership Data The primary dataset encompasses complete 2024 ridership data from Trondheim's 17 active bus routes. Each record represents a single bus arrival at a specific stop, capturing passenger boarding and alighting behavior. Data was provided as 12 monthly CSV files requiring standardized parsing due to inconsistent formatting patterns. Key Processing Steps: • Automated data quality validation (missing values, outliers, temporal consistency) • Standardization

of field naming conventions across monthly files
• Temporal aggregation to hourly intervals for model compatibility

Figure 5.1: Route-by-Route Ridership Distributions

As shown in Figure 5.1, ridership patterns vary significantly across the 17 bus routes, with routes 1, 3, and 11 emerging as the highest-volume services. The extensive outliers visible across all routes indicate significant temporal variability, reflecting the complex interactions between route characteristics, service frequency, and passenger demand patterns.

5.2.2 Weather Data Integration Comprehensive weather data was acquired from the Norwegian Meteorological Institute's Frost API (frost.met.no), implementing rate limiting protocols and comprehensive validation procedures. Hourly weather observations required precise temporal alignment with ridership data and interpolation for missing values.

Figure 5.2: Average Rideship by Temperature Range Figure 5.3: Weather Ridership Correlation Matrix

The correlation analysis (Figure 5.3) reveals modest individual weather correlations with ridership (ranging from -0.020 to 0.009), while strong internal correlations between temperature variables ($r = 0.912$) and wind measurements ($r = 0.897$) indicate multicollinearity considerations for subsequent modeling. Temperature threshold analysis (Figure 5.2) demonstrates a counterintuitive pattern where warmer temperatures (>20°C) correlate with higher ridership, challenging conventional assumptions about weather-induced transit usage patterns. This finding suggests that seasonal activity patterns and tourism effects may outweigh simple weather avoidance behaviors in Trondheim's context.

5.2.3 Event Data Compilation Local events were systematically identified through multiple sources including automated research and manual verification. The event dataset encompasses:

• Sporting Events: Rosenborg BK home games at Lerkendal Stadium • Academic Calendar: University semester periods • Public Holidays: National and regional holiday periods
• Cultural Events: Major concerts, festivals, and larger public gatherings This contextual information provides essential background for understanding ridership anomalies that cannot be explained through temporal or weather patterns alone.

5.3 Data Integration Pipeline The integration process addressed several technical challenges:

5.3.1 Temporal Alignment Synchronizing three datasets with different temporal granularities (transaction-level ridership, hourly weather, daily events) required sophisticated aggregation strategies. Data Quality Management: Implemented comprehensive validation protocols including: • Missing value detection and imputation strategies • Outlier identification using statistical thresholds • Cross-validation between data sources for temporal consistency

5.3.2 Master Dataset Construction The final integrated dataset structure enables multi-level analysis from individual stop performance to system-wide pattern identification, supporting both granular predictions and aggregated insights. The resulting dataset provides comprehensive coverage of passenger movement throughout Trondheim's public transport system, incorporating environmental and contextual factors essential for robust predictive modeling. File: master_dataset_builder.py 6. Interesting Insights The exploratory data analysis discovered several compelling patterns that informed both the feature engineering strategy and model development approach. These insights demonstrate the complex, multi-layered nature of public transportation demand. 6.1 Counterintuitive Weather Effects: Context Matters More Than Conditions The weather analysis uncovered a surprising finding that challenges conventional assumptions about transit behavior. While intuition suggests that unpleasant weather drives people to use public transport, the reality in Trondheim is far more nuanced. This counterintuitive finding aligns with research demonstrating that weather-ridership relationships are more complex than simple modal shift assumptions (Ngo & Bashar, 2024; Zhou et al., 2017).

The Temperature Paradox: System-wide analysis initially showed that warmer temperatures (>20°C) correlate with higher ridership (Figure 6.1), contradicting expectations about weather-induced modal shifts. However, seasonal segmentation revealed the true complexity: during winter months, higher temperatures reduce ridership as people choose walking or cycling over public transport. In summer, the trend reverses - within the commuter subset, ridership is highest on warmest days (>15°C), suggesting that increased city activity during pleasant summer weather is a stronger driver than the incentive to use alternative transport modes. This demonstrates that temperature context is more significant than absolute temperature values. The complex interaction between temperature and ridership reflects the sophisticated seasonality effects documented in suburban transit systems (Kashfi et al., 2015).

Figure 6.1: Impact of Temperature on Ridership

Precipitation Type Hierarchy: The data revealed that frozen precipitation (snow/sleet) is a stronger motivator for transit use than rain. Winter precipitation caused dramatic ridership spikes, while summer rain showed minimal impact. This seasonal context dependency directly informed the decision to engineer interaction features between weather variables and time periods. As seen in Figure 6.2 the precipitation intensity in the winter, affects the average boardings much more significant than in the rest of the year.

Figure 6.2: Impact of Precipitation on Ridership (System-wide)

Figure 6.3: Impact of Precipitation Intensity by Season (Commuters) 6.2 Event Impact: The Power of Granular Analysis System-wide event analysis initially suggested minimal impact, but stop-level analysis revealed the true story.

RBK Home Games - A Tale of Two Scales: • System-wide impact: Only 2.39% increase in average boardings across all stops • Stadium area impact: 270% ridership increase on key routes serving Lerkendal Stadium (visualized in Figure 6.4). • City center spillover: 11.18% increase at major downtown stops from pre/post-game activities

This finding fundamentally changed the modeling approach, demonstrating that meaningful event impacts are highly localized and require stop-level analysis rather than system-wide aggregation.

Figure 6.4: RBK Game day vs. Baseline Day Ridership at Stadium Stops. Holiday Dichotomy: The analysis revealed that public holidays fall into two distinct categories with opposite effects: Celebration-based events (National Day - May 17th): +99.31% ridership increase Absence-based holidays (Christmas, Easter): -38.94% to -51.64% ridership decrease

6.3 The Dominance of Academic Cycles Student semester periods emerged as the single most powerful predictor of ridership variation, creating a 37% system-wide increase during active academic periods. This effect dwarfs all other external factors, including major events and weather conditions. The temporal signature is unmistakable: afternoon ridership (12:00-15:00) increases dramatically during semester periods, reflecting university schedule concentrations. This finding validated the critical importance of academic calendar integration in any Trondheim transit model.

Figure 6.5: Daily Ridership Patterns: Student Semester vs Non-Semester As seen in Figure 6.5 the ridership is significantly higher in the semester seasons. This aligns with Trondheims many academic institutions.

6.4 Route heterogeneity: One Size Does Not Fit All Analysis across Trondheim's 17 bus routes revealed significant operational heterogeneity that directly impacts predictability : High-Volume Central Routes (1, 3, 11): • More stable, predictable patterns • Lower sensitivity to external factors • Consistent ridership with smaller variance • Medium-Volume Residential Routes: • Higher variability and weather sensitivity • Stronger correlation with event impacts • Greater seasonal fluctuation

This heterogeneity suggested that route-specific modeling approaches might outperform system-wide models, though computational constraints led to the unified approach with route ID as a key feature.

Figure 6.6: Ridership Distribution by Route

6.5 Temporal Memory: The Critical Role of Historical Patterns Perhaps the most crucial insight emerged from lag feature analysis: immediate historical ridership is the strongest predictor of future demand. Rolling averages and same-stop previous-hour boardings consistently ranked as top features across all models. This finding validated the transportation engineering principle that transit demand exhibits strong temporal autocorrelation. Passengers create predictable patterns through routine travel behavior, making historical ridership data more valuable than external environmental factors. Modeling Implication: Advanced lag and rolling window features became the project's most impactful enhancement, yielding a 13.75% improvement in model performance.

6.6 Model Algorithm Insights: Why Random Forest Succeeded Comparative analysis of model feature importance revealed fundamentally different learning strategies: Random Forest: Learned nuanced, weather-aware patterns with balanced consideration of route, time, and environmental factors. Six of its top 15 features were weather-related, enabling seasonal adaptability.

XGBoost: Adopted a rule-based approach emphasizing categorical flags (weekend, rush hour, semester) while largely ignoring weather variations. This distinction explained Random Forest's superior cross-validation performance - its continuous, weather-aware approach proved more robust across seasonal transitions than XGBoost's rigid categorical rules. 7. Data Pre-Processing The data pre-processing phase was crucial for transforming the raw data sources into a clean, integrated dataset optimized for machine learning applications. Due to the volume, variety and complexity of the initial data sources, this phase was particularly critical. The comprehensive pipeline addressed critical challenges including data quality issues, memory constraints, temporal alignment and feature standardization across 18.7 million ridership observations. 7.1 Development Environment and AI-Assisted Tools This project was developed using Python 3.11 with standard machine learning libraries including pandas, scikit-learn, matplotlib, and seaborn. To enhance development efficiency and code quality, AI language models (primarily Claude and Gemini) and code utilizing tools like Github Copilot and Gemini Code Assist were utilized as development assistants in the following capacities: • Code debugging and optimization: Assistance with syntax errors, performance improvements, and implementation of best practices for data manipulation and model development • Data preprocessing consultation: Suggestions for handling missing data, feature engineering approaches, and data validation techniques specific to time-series transportation data • Technical documentation: Support for report structure organization and clarity improvements in technical writing • Conceptual clarification: Verification of understanding for complex machine learning concepts during implementation, particularly regarding time-series forecasting methodologies.

All analytical decisions, model selection criteria, hyperparameter tuning strategies, and result interpretations were developed independently. AI tools functioned as sophisticated debugging and consultation resources, similar to Stack Overflow or technical documentation, but with interactive capability for iterative problem-solving. All AI-suggested code

was thoroughly reviewed, tested, and adapted before implementation to ensure understanding and appropriateness for the specific bus ridership prediction use case. This approach reflects modern machine learning development practices where AI assistance enhances productivity while maintaining analytical rigor and domain expertise.

7.1 Raw Data Challenges and Initial Assessment The raw ridership data from AtB, provided in twelve separate monthly CSV files, exhibited inconsistencies in formatting. The first step was to establish a unified schema. A column mapping dictionary was defined to translate the original Norwegian column names to a consistent English standard (e.g., ‘Dato’ to ‘date’, ‘Påstigende’ to ‘boardings’). To ensure all required columns were present, a validation check was performed. To prevent data corruption in the master dataset, files that did not conform, were flagged and skipped.

Mapping all columns for renaming from Norwegian to English for consistency

column_map = { 'Dato': 'date', 'Linje': 'route_id', 'Tur': 'trip_id', 'Holdeplassnummer': 'stop_id', 'Holdeplassekvens': 'stop_sequence', 'Time': 'hour', 'Påstigende': 'boardings' } required_raw_cols = list(column_map.keys())

```
# Processing the first file with a header, then appending the rest
for i, filename in enumerate(all_files):
    print(f"  - Reading and processing {os.path.basename(filename)}...")
    try:
        # reading a single month into temporary DataFrame.
  df = pd.read_csv(filename, sep=';', encoding='utf-8', low_memory=False)
        # Making sure all required columns are present
        if not all(col in df.columns for col in required_raw_cols):
            print(f"    Warning: Skipping file {os.path.basename(filename)} due to missing columns.")
            continue
        # Renaming columns to English.
        df = df.rename(columns=column_map)
        df = df[list(column_map.values())]
```

Code 7.1: Mapping columns for renaming from Norwegian to English.

7.1.2 Data Type Conversion and Missing Value Treatment To ensure consistent data formats and memory efficiency, columns intended to be numeric were explicitly converted. Fields such as trip_id, stop_id and boardings were converted using pd.to_numeric, with the errors=’coerce’ parameter carefully handling any non-numeric entries by transforming them into NaN (Not A Number). A robust handling of missing values strategy was then implemented. Rows that were missing essential identifiers like trip_id and stop_id were dropped, as they could not be reliably used for analysis. Missing values for boardings were imputed with 0, aligning with the operational reality that no passengers were recorded boarding at the specific stop and time.

Converting text columns to numberic, coercing errors to NaN.

for col in ['trip_id', 'stop_id', 'stop_sequence', 'hour', 'boardings']: df[col] = pd.to_numeric(df[col], errors='coerce') # Dropping all rows missing essential information. df.dropna(subset=['trip_id', 'stop_id', 'hour'], inplace=True)

```
        # Converting all clean columns to the most memory-efficient data         types.
        df['boardings'] = df['boardings'].fillna(0).astype('int16')
        df['stop_sequence'] =df['stop_sequence'].fillna(0).astype('int16')
        df['trip_id'] = df['trip_id'].astype('int32')
        df['stop_id'] = df['stop_id'].astype('int32')
        df['hour'] = df['hour'].astype('int8')
        df['route_id'] = df['route_id'].astype('category')
```

Code 7.2: Converting text columns to numeric, coercing errors to NaN(Not A Number). For the weather data, missing values in the hourly observations were addressed through temporal interpolation, using both backward and forward fill methods (bfill().ffill()). The basis for this choice was the logical assumption that weather conditions change gradually and can be reasonably estimated from the neighboring hours. The weather data also needed conversion from UTC+00:00 to local Norwegian time.

Impute weather NaNs for this chunk, using backward and forward fill.

weather_cols = ['temperature_celsius', 'wind_gust_m_s', 'min_temp_day', 'relative_humidity_percent', 'precipitation_mm_hr', 'wind_speed_m_s'] cols_to_fill = [col for col in weather_cols if col in featured_df.columns] for col in cols_to_fill: featured_df[col] = featured_df[col].bfill().ffill() Code 7.3: Imputing weather NaN, using backward and forward fill.

Finally, to manage the large dataset efficiently, data types were optimized for memory usage. Numeric columns were downcast to the smaller appropriate integer types (e.g., int8, int16, int32) and categorical variables like route_id were converted to category dtype, significantly reducing the memory footprint of the dataset. # optimizing data types for large dataset processing df['route_id'] = df['route_id'].astype('category') df['hour_of_day'] = df['datetime'].dt.hour.astype('int8') df['day_of_week'] = df['datetime'].dt.dayofweek.astype('int8') Code 7.4: Optimizing datatypes for large dataset processing.

7.1.3 Outlier Philosophy A formal outlier detection and removal process was deliberately omitted. In a public transport dataset, like this, many statistical outliers do not represent data errors but rather legitimate, high-impact events (e.g. a sudden surge in ridership due to a football game or a big concert). Removing these data points would blind the model to the very events it needs to learn to predict. The chosen modeling approach, tree-based ensembles, is also very robust to the influence of outliers The primary force was therefore on correcting data entry errors (handled by errors=’coerce’) rather than removing valid, but extreme observations. 7.1.4 Event Data Compilation The creation of a comprehensive event dataset was the most complex data gathering tasks, as the information wasn’t readily available from a single, structured source. A hybrid research methodology was employed to ensure accuracy and completeness.

An initial broad search for significant local events in Trondheim during 2024 was conducted using Large Language Models(LLMs). This automated approach provided a wide net to summon major festivals, concerts and public gatherings. All events identified by the LLMs were then subject to rigorous manual verification, where official sources such as municipal calendars, university schedules and event listing websites were cross-referenced to confirm date and scope. To make the data useful for the model, the events were systematically categorized into seven distinct types, including is_rbk_home_game, is_major_event and is_student_semester. This allowed the model to learn different patterns of ridership impact associated with the different types of events. All verified events, which were initially defined by daily dates, were converted to an hourly resolution to match the granularity of the ridership and weather data. This was achieved by creating a complete hourly timeline for an entire year and applying a True flag to each hour that fell within an event’s duration. For example, the is_rbk_home_game flag was set to True for a five-hour window (two hours before, the two hours of, and one hour after kickoff) on match days to capture the full scope of fan-related travel. 7.2 Data Integration and Temporal Alignment The analytical power of the project comes from the integration of ridership, weather and event data. This required precise temporal alignment. 7.2.1 Temporal Standardization By combining the separate date and hour columns in the ridership dataset, a unified datetime column was created. This standardized timestamp served as the primary key for merging all the data sources.

```
# Creating a unified datetime column.
```

df['datetime'] = pd.to_datetime(df['date'] + ' ' + df['hour'].astype(str) + ':00:00', errors='coerce') df.dropna(subset= ['datetime'], inplace=True) df.drop(columns=['date', 'hour'], inplace=True) Code 7.5: Creating unified datetime column

To ensure the compatibility, any timezone information was removed from the datetime columns in the weather and the event datasets (.dt.tz_localize(None)) before merging. weather_df['datetime'] = pd.to_datetime(weather_df['datetime']).dt.tz_localize(None) events_df['datetime'] = pd.to_datetime(events_df['datetime']).dt.tz_localize(None)

Merging and then engineering features for this month only

merged_df = pd.merge(df, weather_df, on='datetime', how='left') merged_df = pd.merge(merged_df, events_df, on='datetime', how='left') Code 7.6: Removing timezone information from datetime columns in the weather and events datasets.

7.2.2 Chunked Processing for Memory Management The dataset size of 18.7 million records was not compatible to load and process all files at once, with the available hardware. To overcome this challenge, a chunked processing strategy was implemented. The twelve monthly ridership CSV files were read and processed sequentially in chronological order. After each month’s data was cleaned and feature-engineered, it was appended to the master dataset file. This iterative approach ensured that the entire dataset could be processed due to the hardware constraints, and that the chronologically integrity required for time-series analysis, was preserved. # ---Processing Raw Ridership Data and Writing to CSV in Chunks --- all_files = sorted(glob.glob(os.path.join(RIDERSHIP_DIR, "*.csv"))) # Sort files to process chronologically if not all_files: raise FileNotFoundError(f"No CSV files found in directory: ")

```
# Writing the month to the final CSV
 write_mode = 'w' if i == 0 else 'a'
```

```
        write_header = True if i == 0 else False

        print(f"    - Writing {len(featured_df)} processed rows to CSV...")
        featured_df.to_csv(FULL_OUTPUT_PATH, index=False, mode=write_mode, header=write_header)
```

Code 7.7: Processing raw ridership data and writing to CSV in chunks.

Crucially, all time-sensitive feature engineering (such as lag and rolling window features, discussed in section 9) was performed after the complete, chronologically sorted master dataset was assembled. This ensures that calculations at the boundaries between months (e.g. first hour of February 1.) had access to data from the last hours of January 31th, thereby preventing data leakage and preserving the dataset's temporal integrity. 7.3 Foundational Feature Engineering

7.3.1 Temporal Feature Creation To allow the model to learn time-dependent patterns, several features were derived directly from the datetime column. These included hour_of_day, day_of_week, day_of_year and month. Binary flags were also created to explicitly identify is_weekend and is_rush_hour periods, making sure to capture key behavioural patterns in public transport usage. def engineer_features_for_chunk(df: pd.DataFrame) -> pd.DataFrame: """ Engineers the set of chosen features for a given DataFrame chunk. """ print(" - Engineering features for the current month...")

```
# Temporal Features
df['hour_of_day'] = df['datetime'].dt.hour.astype('int8')
df['day_of_week'] = df['datetime'].dt.dayofweek.astype('int8')
df['day_of_year'] = df['datetime'].dt.dayofyear.astype('int16')
df['month'] = df['datetime'].dt.month.astype('int8')
df['is_weekend'] = df['day_of_week'].isin([5, 6])
df['is_rush_hour'] = (df['hour_of_day'].isin([7, 8, 15, 16])) & (~df['is_weekend'])
```

return df Code 7.8: Engineering features for chunk-loading

The creation of domain-specific temporal features is supported by research demonstrating the importance of capturing multiple time scales in transit demand prediction (Kandiri et al., 2024, Baghani et al., 2023).

7.3.2 Route-Specific Operational Features To provide the model with the context of a stop's location during a route, operational features were engineered. These included binary flags to identify the is_first_stop and is_last_stop of a trip. Furthermore, a stop_sequence_normalized feature was created by dividing the stop sequence number by the maximum sequence for that trip, providing a relative measure of progress along the route from 0 to 1.

```
# Route specific features
```

max_sequence_per_trip = df.groupby(['route_id', 'trip_id'], observed=False)['stop_sequence'].transform('max') df['is_first_stop'] = df['stop_sequence'] == 0 df['is_last_stop'] = df['stop_sequence'] == max_sequence_per_trip df['stop_sequence_normalized'] = (df['stop_sequence'] / max_sequence_per_trip).fillna(0).astype('float32') Code 7.9: Creating route specific feature: stop_sequence_normalized

7.4 Analytical Quality Assurance To ensure that the quality of the data was assured, a data diagnostic tool script was created to perform a "sanity check" of the processed data. For instance, it included functions to verify that all 12 months of the data were present in the final dataset, a check that proved critical in identifying an initial data corruption issue with one of the monthly CSV files, October, where the dates was from January. The tool also allowed for spot-checking ridership patterns for specific stops and routes, ensuring that the data logically reflected real-world operational realities before being used for model training. This analytical validation provided an essential layer of confidence in the quality of the foundation dataset. 7.5 Final Dataset Characteristics The comprehensive pre-processing pipeline concluded with a series of data quality assurance checks, including verifying temporal consistency and validating ridership patterns against known operational constraints. The final, analysis-ready dataset consisted of 18.704.349 individual stop arrival records across 17 routes and 847 unique stops. This foundational dataset, with its 24 engineered features, was ready for the machine learning modeling phase. 8. Measuring Performance This section outlines the evaluation metrics used to assess model performance and the methodology for comparing different algorithms.

Evaluation Metrics: • Mean Absolute Error (MAE): Measures average absolute difference between predictions and actual values • Root Mean Square Error (RMSE): Penalizes larger errors more heavily • R-squared (R^2): Indicates proportion of variance explained by the model • Mean Absolute Percentage Error (MAPE): Provides percentage-based error metric

All metrics are reported to 2 decimal places for consistency throughout this report.

8.1 Primary Metrics Comparison 8.1.1 Mean Absolute Error (MAE) as the Primary Metric Mean Absolute Error was selected as the primary evaluation metric due to its direct interpretability in the business context and operational relevance for transportation planning. MAE, unlike squared error metrics, provides a linear, intuitive measure of prediction accuracy that corresponds directly to passenger counts. This makes it immediately understandable to operational staff responsible for resource allocation. As argued by Willmott & Matsuura (2005), MAE is a more natural and unambiguous measure of average error than the commonly used Root Mean Squared Error (RMSE). The business value of MAE lies in its straightforward interpretation. A MAE of 1.38 passengers, for example, means that, on average, the model's predictions deviate from actual ridership by 1.38 passengers. This enables operational planners to understand

prediction uncertainty in terms relevant to capacity planning, service frequency decisions and resource deployment strategies. Furthermore, MAE's robustness to outliers was considered valuable for this dataset (Hodson, 2022). During operational disruptions such as weather delays or special events like RBK home games, ridership can spike dramatically or drop unexpectedly. These extreme values could skew squared error metrics, making models appear worse than they actually are in normal conditions. By treating all errors linearly, MAE provides a more stable and reliable measure of day-to-day prediction accuracy. 8.1.2 Supplementary Performance Indicators While MAE served as the primary metric, a suite of supplementary indicators was used to provide a multi-dimensional assessment of the model's capabilities and limitations.

Coefficient of Determination (R^2) R-squared was used to measure how much of the ridership variability the models could actually explain, providing insight into whether they were capturing real patterns or just memorizing noise. The metric indicates how well the model understands the underlying patterns in bus ridership (Chicco et al,.2021). The higher the R^2, the more the model has learned genuine relationships between factors like weather, time of day and passenger demand, rather than fitting random fluctuations in the data.

Root Mean Square Error (RMSE) To provide a measure of error that is sensitive to large deviations, RMSE was calculated. As it heavily weights large errors, it proved useful for identifying the impact of outliers and extreme prediction scenarios that could affect service quality.

```
1. from sklearn.metrics import mean_absolute_error, r2_score
2.
3. Def evaluate_predictions(y_true, y_pred, fold_name):
4.  """
5.  Comprehensive evaluation of predictions
6.  """
7.  mae = mean_absolute_error(y_true, y_pred)
8.  rmse = np.sqrt(mean_squared_error(y_true, y_pred))
9.  r2 = r2_score(y_true, y_pred)
10. mape, mae_zeros = safe_mape(y_true, y_pred)
11.
12. results = {
13.     'fold': fold_name,
14.     'mae': mae,
15.     'rmse': rmse,
16.     'r2': r2,
17.     'mape_non_zero': mape,
18.     'mae_zeros': mae_zeros,
19.     'n_samples': len(y_true)
20. }
21.
22. Return results Code 8.1: Establishing evaluation metrics.
```

Custom "Safe" Mean Absolute Percentage Error (Safe-MAPE) A significant challenge in transportation datasets is the frequent occurrence of zero-boarding events, which makes the traditional Mean Absolute Percentage Error (MAPE) computationally undefined. To address this, a custom safe_mape function was developed:

This function provides a complete picture of model performance by splitting evaluation into two distinct parts: • MAPE on non-zero instances: Measuring the average percentage error when passengers are actually present • MAE on zero instances: Measuring how many "ghost" passengers the model incorrectly predicts when stops are empty 23. def safe_mape(y_true, y_pred): 24. "*Calculates a safe version of the Mean Absolute Percentage Error (MAPE)." 25. mask = y_true != 0 26. mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100 27. mae_zeros = np.mean(np.abs(y_true[~mask] - y_pred[~mask])) 28. print(f" - MAPE on non-zero boardings: %") 29. print(f" - MAE on zero boardings: ") 30. return mape, mae_zeros Code 8.2: Calculating a safe version of the Mean Absolute Percentage Error due to many records with zero boardings.*

8.2 Cross-Validation Strategy To ensure robust and realistic evaluation of model performance, a comprehensive time-series cross-validation framework was established and applied at every stage of model development and selection. This methodology follows established best practices for time series evaluation (Cerqueira et al., 2020), which demonstrates that blocked cross-validation significantly outperforms standard randomized k-fold approaches for temporal data by preserving temporal ordering and preventing data leakage. 8.2.1 Need for Time-Series Specific Approach For time-dependent data like bus ridership, standard cross-validation methods such as k-fold are invalid, as they would involve training a model on future data to predict the past, known as data leakage, leading to unreliable performance estimates. As argued by Tashman (2000), out-of-sample testing that preserves the temporal order of data is essential for meaningful evaluation of forecasting models. Therefore, a "walk-forward" validation methodology was adopted, ensuring that models are always trained on historical data and tested on unseen future data, mimicking real operational deployment. 8.2.2 The Expanding Window Cross-Validation Framework The chosen validation framework was a 6-fold expanding window cross-validation. This method involves splitting the dataset into six sequential folds, where the training period progressively grows, and the model is tested on the subsequent month:

Cross-validation parameters

SAMPLE_SIZE_PER_FOLD = 800000 # Sample size for training per fold RANDOM_STATE = 42

Define the 6 time series folds (expanding window)

cv_folds = [{'train_start': '2024-01-01', 'train_end': '2024-02-28', 'test_start': '2024-03-01', 'test_end': '2024-03-30'}, {'train_start': '2024-01-01', 'train_end': '2024-04-29', 'test_start': '2024-05-01', 'test_end': '2024-05-31'}, {'train_start': '2024-01-01', 'train_end': '2024-06-29', 'test_start': '2024-07-01', 'test_end': '2024-07-31'}, {'train_start': '2024-01-01', 'train_end': '2024-08-30', 'test_start': '2024-09-01', 'test_end': '2024-09-30'}, {'train_start': '2024-01-01', 'train_end': '2024-10-30', 'test_start': '2024-11-01', 'test_end': '2024-11-30'}, {'train_start': '2024-01-01', 'train_end': '2024-11-29', 'test_start': '2024-12-01', 'test_end': '2024-12-31'},] Code 8.3: Creating custom time-series cross-validation folds

Figure 8.1 clearly shows the expanding nature of the training data and the sequential, non-overlapping test sets.

8.2.3 Validation Logic and Implementation The cross-validation implementation ensured temporal integrity through careful data partitioning: for fold_idx, fold in enumerate(cv_folds): print(f"\n{'='*20} FOLD {fold_idx + 1}/{len(cv_folds)} {'='*20}") print(f"Training: {fold['train_start']} to {fold['train_end']}") print(f"Testing: {fold['test_start']} to {fold['test_end']}")

Create train/test masks

```
train_mask = (
    (full_df['datetime'] >= fold['train_start']) &
    (full_df['datetime'] <= fold['train_end'])
)
test_mask = (
    (full_df['datetime'] >= fold['test_start']) &
    (full_df['datetime'] <= fold['test_end'])
)
```

Code 8.4: Ensuring temporal integrity through careful partitioning.

Each fold maintained strict temporal separation, ensuring no data leakage between training and test periods.

8.2.4 Performance Stability Assessment The 6-fold approach enabled assessment of model stability across different temporal contexts. Performance variability was measured using standard deviation across folds, providing crucial insight into model robustness for operational deployment.

8.2.5 Business Relevance of Validation Design This validation strategy directly addresses practical deployment considerations for AtB. The expanding window approach mirrors the reality of operational forecasting, where models trained on historical data must predict demand for upcoming periods. The monthly evaluation periods align with AtB's operational planning cycles, ensuring that validation results correspond to actual decision-making timeframes.

8.3 Business-Relevant Evaluation While statistical metrics like MAE and R^2 are essential for evaluating model accuracy, a successful forecasting project must also be evaluated on its ability to deliver tangible operational value. The evaluation framework therefore extends to include criteria that directly address AtB's operational priorities.

8.3.1 Computational Performance Before deployment, it is critical to ensure a model is computationally feasible. Training and prediction times were systematically tracked to assess operational viability: # Train the model import time

start_time = time.time() model.fit(X_train_model, y_train_fold) training_time = time.time() - start_time print(f" Training time: seconds")

```
# Predict on test set
start_time = time.time()
y_pred_fold = model.predict(X_test_model)
prediction_time = time.time() - start_time
print(f" Prediction time: {prediction_time:.2f} seconds")
```

Code 8.5: Tracking and outputting training and prediction time.

Model Training Time (sec) MAE (std) R2 (std) Computational Efficiency Linear Regression 2.18 1.99 (0.15) 0.06 (0.012) Fastest training XGBoost 10.26 1.77 (0.12) 0.30 (0.012) Good Balance Random Forest 292.74 1.60 (0.10) 0.31 (0.03) Slower but most accurate Figure: 8.2: Table with Computational Performance Results

All models demonstrated exceptional prediction speed, generating forecasts for over 800,000 data points in under 10 seconds, comfortably meeting AtB's operational requirements for updates every 15-30 minutes.

8.3.2 Model Evolution Performance Summary The systematic development approach yielded substantial improvements:

Development Stage MAE (±std) R^2 (±std) Training Time Key Enhancement Baseline Random Forest 1.60 (±0.10) 0.31 (±0.03) 292.74 sec Initial benchmark Advanced Feature Engineering 1.38 (±0.13) 0.49(±0.016) 63.67 sec Lag/rolling features Express Hyperparameter Tuning 1.48(±0.09) 0.372(±0.055) 6.97 min tota l(tuning) Optimized parameters Final Model (Oct-Dec Holdout) 1.38 0.52 [TBD] sec Complete pipeline Figure 8.3: Model Evaluation Table Key Performance Insights: Feature Engineering Impact: Advanced temporal features yielded the most significant improvement (13.75 % MAE reduction), demonstrating that domain-informed feature creation is more impactful than algorithm selection. Hyperparameter Tuning Limitations: Express tuning provided modest gains while revealing computational constraints - a critical real-world finding about diminishing returns. Model Stability: Random Forest demonstrated superior consistency (lowest standard deviation) across seasonal variations, crucial for reliable operational deployment. Having established the evaluation framework and documented the 13.75% improvement, this section examines the error patterns underlying this performance to understand where and when the model succeeds or struggles.

8.3.3 Model Error Analysis and Performance Validation 8.3.3.1 Temporal Error Pattern Analysis Analysis of prediction errors across different hours reveals systematic patterns crucial for operational deployment strategies. The temporal variation shows a 3.6x difference in MAE across hours, from 0.5 passengers during early morning (4-6 AM) to 1.8 passengers during peak periods (7-8 AM and 14-15 PM). Early morning operations (4-6 AM) demonstrate the model's most accurate predictions with MAE ranging from 0.5-1.0 passengers. The low ridership volumes create predictable, stable patterns suitable for precise scheduling decisions. Conversely, peak periods (7-8 AM, 14-15 PM) exhibit the highest prediction errors of 1.6-1.8 passengers, reflecting increased ridership variability and commuter pattern complexity. Evening hours (19-23 PM) show decreasing MAE from 1.4 to 1.0 passengers, demonstrating the model's ability to capture consistent evening ridership patterns. This temporal variation indicates that time-of-day significantly impacts prediction reliability, suggesting operational deployment should implement hour-specific confidence intervals and dynamic capacity buffers adjusted for historical error patterns. 8.3.3.2 Route-Level Error Heterogeneity The analysis across Trondheim's 17 bus routes reveals substantial heterogeneity that directly impacts deployment strategy. Route performance shows a 4x variation in prediction accuracy, ranging from MAE of 0.5 passengers on suburban routes to 2.1 passengers on central routes. High-performance routes (34, 20, 15, 23) achieve MAE between 0.5-1.0 passengers, characterized by lower ridership volumes and predictable suburban patterns. These routes are immediately suitable for precise capacity planning and automated scheduling decisions. Medium-performance routes (10-14) demonstrate moderate prediction accuracy with MAE of 1.0-1.5 passengers, suitable for operational deployment with standard monitoring protocols. Challenging routes (1, 2, 3, 28) exhibit MAE exceeding 2.0 passengers, representing high-volume central routes with complex ridership patterns. Route 3 shows the highest MAE (~2.1), indicating maximum prediction difficulty. These operationally critical routes require enhanced monitoring and conservative capacity planning with manual oversight capabilities. The route heterogeneity reveals that high-volume central routes that are most operationally important also present the greatest prediction challenges, necessitating route-specific deployment strategies and risk mitigation protocols. 8.3.3.4 Stop-Level Performance Insights Analysis of prediction accuracy at individual stops reveals geographic patterns that inform network-level operational strategies. The 76x variation between best and worst performing stops demonstrates significant spatial heterogeneity in prediction reliability. Best performing stops include Ranheim (MAE: 0.08), Marienborg stasjon 3 (MAE: 0.09), and Mikkelvegen (MAE: 0.01), primarily suburban/residential locations with consistent low-to-moderate ridership patterns. These locations enable precise micro-level capacity planning decisions. Most challenging stops are concentrated in Trondheim's city center, with Kongens gate K1 (MAE: 5.8), Ladeveien (MAE: 5.04), and Kongens gate K2 (MAE: 5.0) showing the highest prediction errors. These high-traffic interchange points experience complex ridership dynamics that challenge unified modeling approaches. The stark contrast between suburban accuracy and urban center difficulty suggests that prediction challenges correlate with urban density and transit complexity, supporting implementation of location-specific confidence intervals and enhanced monitoring protocols for city center operations. 8.3.3.5 Special Event Analysis: RBK Game Impact Analysis of Rosenborg BK home game impacts reveals counter-intuitive ridership patterns that demonstrate the model's sophisticated pattern recognition capabilities. Rather than simple ridership increases, the model learned systematic ridership reductions on central routes during game days: Route 1 (15.3% decrease), Route 2 (12.0% decrease), and Route 11 (23.2%

decrease). Only Route 10 showed increased ridership (8.5%). This unexpected pattern suggests that regular commuters avoid peak travel times during major events due to anticipated congestion, validating the importance of event feature engineering and demonstrating that the model successfully distinguishes between event-driven and baseline demand patterns. 8.3.3.6 Zero-Inflation Handling Assessment Given that approximately 66% of bus stop arrivals result in zero boardings, the model's handling of this zero-inflation challenge represents a critical performance dimension. Analysis of consistently low ridership stops reveals the model's effectiveness in predicting sparse demand scenarios, with accurate predictions for stops like Ranheim (predicted 0.04, actual 0.01) and Marienborg stasjon 3 (predicted 0.06, actual 0.002). The model's ability to achieve 1.38 MAE while accurately predicting extremely low ridership values demonstrates sophisticated distinction between true zeros and small positive values, validating the effectiveness of the lag feature and rolling average approach. 8.3.3.7 Business Impact and Deployment Readiness The comprehensive error analysis validates that the 13.75% improvement represents genuine algorithmic advancement rather than statistical coincidence. The systematic patterns observed across temporal, route, and geographic dimensions demonstrate meaningful ridership relationship learning. The 1.38 MAE translates to operational uncertainty of ±1-2 passengers per prediction, representing approximately 10-15% relative error for medium-volume routes and 30-50% for low-volume routes. For AtB's operations, this means bus capacity planning remains reliable (40-seat buses can accommodate the prediction variance), while enabling 15-20% more precise resource allocation compared to current scheduling methods Immediate deployment applications include suburban route planning (MAE <1.0), off-peak scheduling (declining evening errors), and low-demand stop optimization using accurate zero-inflation predictions. Enhanced monitoring requirements apply to central routes (requiring 2-3 passenger capacity buffers), peak hour operations (1.6-1.8 MAE), and city center stops (5+ passenger prediction uncertainty). The model's conservative over-prediction bias in extreme cases provides a natural safety margin estimated at 1-2 additional passengers during high-uncertainty scenarios, operationally favorable for passenger service quality while manageable for resource allocation efficiency. The error analysis confirms that the developed model provides substantial operational value within understood limitations, positioning AtB to achieve immediate efficiency gains while building toward more sophisticated predictive capabilities. The comprehensive metric framework enables multi-dimensional model assessment aligned with transportation operational priorities. MAE serves as the primary optimization target for direct operational interpretability, while R^2 provides fundamental capability assessment. The combination ensures that model selection and optimization processes align with real-world deployment requirements while maintaining statistical rigor appropriate for academic and professional evaluation standards.

9. Algorithms Applied Three machine learning algorithms were systematically evaluated for bus ridership prediction: Random Forest, XGBoost, and Neural Linear Regression. Each algorithm was chosen for its specific strengths in handling the complex temporal and multivariate nature of transportation data.

9.1 Random Forest Algorithm 9.1.1 Algorithm Overview Random Forest is a powerful ensemble learning method that operates by constructing a forest of decision trees at training time and outputting the average prediction of these trees. It is a refinement of "bagging" (Bootstrap Aggregating) that also incorporates feature randomness, helping to de-correlate the trees and reduce variance, making it robust against overfitting. Random Forest was selected as the baseline model due to its robustness, interpretability and strong performance on tabular data. It often achieves strong results with minimal hyperparameter tuning, and can naturally capture complex, non-linear relationships and feature interactions without explicit feature engineering. Additionally, it is less sensitive to outliers than many other models and can handle mixed feature types. For ridership prediction, these characteristics are particularly valuable given the complex interactions between temporal, weather and operational factors. Random Forest's effectiveness in bus transportation prediction has been empirically validated (Huang et al., 2022), demonstrating superior performance in capturing nonlinear seasonal characteristics.

9.1.2 Implementation and Performance The model was implemented using the RandomForestRegressor class from Scikit-learn:

Implementing Random Forest in the CV loop.

From sklearn.ensemble import RandomForestRegressor

Initialize the model once before the loop

```
print("\nTraining Random Forest...")
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1, # Using all available CPU cores
    min_samples_leaf=5, # Reduces overfitting
  max_depth=15,
    oob_score=True # Using out of bag samples to estimate R2 on unseen data
)
# Inside the loop for each fold
rf_model.fit(X_train_fold, y_train_fold)
```

```
print("Making predictions...")
y_pred_fold = rf_model.predict(X_test_fold)
```

Code 9.1: Implementing Random Forest in the CV loop.

The Random Forest baseline achieved a Mean MAE of 1.60 and a Mean R^2 of 0.31. It also demonstrated high stability, with a low standard deviation in its performance across the six cross-validation folds, serving as the primary benchmark to beat.

Figure 9.1 Top 15 most important features for the Random Forest baseline model.

9.2 XGBoost (Extreme Gradient Boosting) 9.2.1 Algorithm Overview XGBoost is an advanced and highly efficient implementation of the gradient boosting algorithm. Unlike Random Forest, which builds trees independently, gradient boosting builds them sequentially, with each new tree trained to correct the errors of the previous ones. XGBoost optimizes this process with advanced regularization and algorithmic efficiencies. The superiority of XGBoost in transportation contexts is well-documented, achieving R2 = 98.7% in metro ridership prediction (AlKhereibi et al., 2023) and demonstrating effectiveness with temporal feature engineering (Zhang et al., 2022). The algorithm was selected to test whether a state-of-the-art boosting method could outperform the Random Forest baseline. ‘ 9.2.2 Implementation and Performance

Implementing XGBoost in the CV loop

Import xgboost as xgb

Initializing the model

xgb_model = xgb.XGBRegressor(n_estimators=200, max_depth=3, learning_rate=0.1, subsample=0.9, colsample_bytree=0.9, reg_alpha=0.1, reg_lambda=2.0, objective='reg:squarederror', random_state=RANDOM_STATE, n_jobs=-1, enable_categorical=True)

Inside the loop for each fold

xgb_model.fit(X_train_fold, y_train_fold) print("Making predictions...") y_pred_fold = xgb_model.predict(X_test_fold) Code 9.2: Implementation of XGBoost

XGBoost achieved a Mean MAE of 1.77 and a Mean R^2 of 0.30, performing slightly worse than the Random Forest model in terms of average error but demonstrating significantly faster training times

9.3 Linear Regression 9.3.1 Algorithm Overview Linear regression is a fundamental statistical and machine learning algorithm that models the relationship between a dependent variable (boardings) and independent variables (features) by fitting a linear equation to the observed data. The model learns coefficients for each feature, representing the feature's contribution to the prediction. The model was included for comparison to establish the performance of a simple linear approach versus more complex tree-based methods. While the complexity of ridership patterns suggested a linear model would be insufficient, including it provides important context for understanding the value of non-linearity.

9.3.2 Implementation and Performance

Implementing LinearRegression in the CV loop

From sklearn.linear_model import LinearRegression

Scaling features for Linear Regression

scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test)

Linear Regression model

linear_model = LinearRegression() linear_model.fit(X_train_fold, y_train_fold) Y_pred_fold = linear_model.predict(X_test_fold) Code 9.3: Implementation of Linear Regression The model's performance was, as

expected, limited. It achieved a Mean MAE of 1.99 (±0.15) and a Mean R^2 of 0.06, confirming that the relationships in the data are highly non-linear and require more sophisticated models.

9.4 Model Comparison and Selection The performance of all three algorithms was compared across the 6-fold time-series cross-validation. The key criteria for comparison were predictive accuracy (Mean MAE, Mean R^2), performance stability (Standard Deviation of MAE), and computational efficiency.

Model Mean MAE (+/-std.dev) Mean R2 (+/- std.dev) Avg. Training time (s) Linear Regression 1.99 (0.06) 0.06 (0.012) 2.18 Random Forest 1.60 (0.10) 0.31 (0.035) 292.74 XGBoost 1.77 (0.12) 0.30 (0.012) 10.26 Figure 9.2: Comparative Model Performance Across 6-fold Time-Series Cross-Validation (Note: Training times are based on the comparative cross-validation with a sample size of 1000000 per fold). Figure 9.3: A comparative box plot of Mean Absolute Error (MAE) distributions across the six cross-validation folds. The plot highlights the superior and more stable performance of Random Forest compared to the other algorithms. The box plot clearly illustrates that the Random Forest model not only achieves the lowest median MAE, but also has the tightest interquartile range, indicating the most consistent and stable performance across the different seasonal periods tested.

9.4.1 Discussion of Results and Champion Selection The comparative analysis demonstrates that both ensemble methods vastly outperform Linear Regression, confirming the non-linear nature of the problem. The choice between Random Forest and XGBoost presents a classic trade-off between operational factors. XGBoost offers a substantial advantage in computational efficiency, with average training time of approximately 10 seconds per fold compared to almost 5 minutes for Random Forest. In real-world scenarios requiring frequent model retraining, this speed advantage could be operationally significant. However, the primary objective was to develop the most accurate and reliable forecasting tool possible. Random Forest achieved superior average performance (MAE 1.60 vs 1.77) and demonstrated greater stability, evidenced by lower standard deviation in MAE scores (0.10 vs 0.12). This consistency across different seasonal periods is critical for systems supporting reliable operational planning. Given that predictive reliability is paramount for public transport operators, the combination of superior accuracy and consistency makes Random Forest the more robust choice, outweighing XGBoost's computational speed advantage. Random Forest was selected as the champion model for subsequent enhancement through advanced feature engineering and hyperparameter optimization.

9.5 Advanced Feature Engineering Impact With Random Forest established as the champion algorithm, the next phase focused on enhancing model performance through domain-informed feature engineering. The baseline Random Forest achieved a Mean MAE of 1.60 (±0.10 std.dev) across 6-fold time-series cross-validation, providing a robust foundation for systematic enhancement. 9.5.1 Temporal Feature Engineering Strategy The advanced feature engineering approach was designed to provide the model with "temporal memory" - the ability to leverage historical ridership patterns critical for accurate transportation demand forecasting. Three categories of advanced features were systematically implemented: • Lag Features: Capturing immediate and weekly temporal dependencies • Rolling Window Features: Incorporating short-term historical trends • Cyclical Temporal Features: Encoding time-based periodicities

9.5.2 Implementation Methodology The feature engineering pipeline was implemented using the same rigorous 6-fold expanding window cross-validation framework to ensure fair comparison with the baseline model. Critical data sorting procedures were implemented to prevent temporal leakage:

Sort values correctly to ensure accurate lag/rolling calculations

This is a critical step for time-series features

full_df.sort_values(['stop_id', 'route_id', 'datetime'], inplace=True) Code 9.4: Sorting values for time-series features 9.5.2.1 Lag Features Implementation Two strategically selected lag features were created to capture different temporal dependencies:

Group by stop and route to calculate previous hour's boardings

full_df['boardings_same_stop_last_hour'] = full_df.groupby(['stop_id', 'route_id'], observed=True) ['boardings'].shift(1).fillna(0)

Group by stop, route, and hour to get boardings from the same hour a week ago

full_df['boardings_same_hour_last_week'] = full_df.groupby(['stop_id', 'route_id', 'hour_of_day'], observed=True) ['boardings'].shift(168).fillna(0) Code 9.5: Creating Lag features The one-hour lag captures immediate demand continuation, reflecting scenarios where high ridership typically indicates sustained demand. The weekly lag captures commuting routines and recurring travel patterns fundamental to public transportation usage. 9.5.2.2 Rolling Window Features Three-hour rolling statistics were calculated to provide short-term trend awareness:

Group for rolling calculations

grouped = full_df.groupby(['stop_id', 'route_id'], observed=True)['boardings']

Calculate rolling mean and max over a 3-hour window from the past

full_df['boardings_rolling_mean_3h'] = grouped.transform(lambda x: x.shift(1).rolling(3, min_periods=1).mean()).fillna(0) full_df['boardings_rolling_max_3h'] = grouped.transform(lambda x: x.shift(1).rolling(3, min_periods=1).max()).fillna(0) Code 9.6: Creating Rolling window features. These features enable the model to distinguish between normal operational periods and emerging high-demand situations, crucial for proactive service adjustments. 9.5.2.3 Cyclical Temporal Features Traditional hour-of-day and day-of-week categorical features were enhanced with cyclical encoding to capture the continuous nature of time:

--- Cyclical Time Features ---

print(" Creating cyclical time features...") full_df['hour_sin'] = np.sin(2 * np.pi * full_df['hour_of_day'] / 24) full_df['hour_cos'] = np.cos(2 * np.pi * full_df['hour_of_day'] / 24) full_df['day_of_week_sin'] = np.sin(2 * np.pi * full_df['day_of_week'] / 7) full_df['day_of_week_cos'] = np.cos(2 * np.pi * full_df['day_of_week'] / 7) Code 9.7: Creating Cyclical time features. This encoding ensures that 23:00 and 01:00 hours are recognized as temporally adjacent, improving the model's understanding of daily cycles.

9.5.3 Performance Results The Random Forest model with advanced feature engineering demonstrated substantial improvement across all evaluation metrics:

Metric BaselineRF AdvancedRF Improvement Mean MAE 1.60 (±0.10) 1.38 (±0.13) 13.75% Mean R2 0.31 (±0.035) 0.49 (+0.016) 56.5% Training Time (sec) 292.74 63.67 Improved Efficiency Figure 9.3: Table showing the Improvement in metrics and percentage of Random Forest Model with Advanced Feature Engineering over Baseline.

The 13.75% reduction in Mean Absolute Error represents the most significant single improvement achieved in the project, reducing prediction error by approximately 0.21 passengers per prediction. The substantial improvement in R^2 (from 0.31 to 0.49) indicates that advanced features enabled the model to explain nearly 49% of ridership variance, compared to 31% for the baseline.

9.5.4 Feature Importance Analysis Feature importance analysis revealed that newly engineered temporal features achieved dominant positions in the model's decision-making process, validating the effectiveness of domain-informed feature engineering. Temporal Features Dominance The advanced temporal features captured four of the top five most important positions: • Boardings_rolling_mean_3h: 0.23 (highest importance) • Boardings_rolling_max_3h: 0.19 (second highest) • Boardings_same_stop_last_hour: 0.16 (third highest) • Boardings_same_hour_last_week: 0.09 (fifth highest) Combined, these four engineered features account for approximately 67% of the model's predictive power, demonstrating that historical ridership patterns are the strongest indicators of future demand.

Figure 9.2: Feature Importance Comparison (Baseline vs. Advanced Features)

9.6 Hyperparameter Optimization With the advanced feature-engineered Random Forest achieving strong performance (MAE of 1.38), the final optimization phase investigated whether systematic hyperparameter tuning could extract additional predictive accuracy. However, computational constraints necessitated development of a pragmatic "express tuning" approach that balanced optimization potential with practical resource limitations. 9.6.1 Computational Challenges and Strategic Response Initial attempts at comprehensive hyperparameter optimization using traditional grid search and randomized search methods proved computationally prohibitive. The combination of the large dataset (18.7

million records), 6-fold time-series cross-validation, and extensive parameter grids required computational resources exceeding available hardware capabilities. Rather than abandoning optimization entirely, an innovative "express tuning" strategy was developed to achieve meaningful improvements within strict computational constraints: Express Tuning Configuration: Reduced validation scope: 3 folds instead of 6 for faster evaluation Strategic sampling: 800,000 most recent records per training fold to maintain temporal relevance Focused parameter search: 8 randomly selected promising combinations from a reduced grid

9.6.2 Express Tuning Results The express tuning approach employed a manual optimization loop testing 8 strategically selected parameter combinations across 3 temporal folds. Each parameter set was evaluated using the same expanding window methodology as previous experiments, ensuring consistency in evaluation standards. Optimal Hyperparameters Identified: • n_estimators: 150 (balanced complexity and performance) • max_depth: 12 (sufficient for complex patterns without overfitting) • min_samples_leaf: 50 (regularization for generalization) • max_features: 'log2' (optimal feature subsampling)

9.6.3 Final Model Performance The optimized Random Forest model was evaluated using the express tuning methodology and subsequently tested on the October-December 2024 holdout set: Model Stage MAE R2 Improvement Computational Time Advanced Features RF 1.38 (±0.13) 0.49 (±0.016) - 63.67 sec avg Express Hyperparameter Tuning 1.48 (±0.098) 0.37 (±0.055) - 6.97 minutes total Final Model (Holdout Test) 1.38 0.51 Minimal Change -

9.6.4 Performance Analysis and Key Findings The final holdout performance (MAE: 1.38) validates that the advanced feature engineering had achieved near-optimal performance for the Random Forest algorithm. The express hyperparameter tuning provided minimal improvement over the feature-engineered model, demonstrating a critical finding: domain-informed feature engineering provides far greater performance gains than algorithmic optimization. Key Insights:

1. Feature Engineering Supremacy: The 13.75% improvement from advanced features far exceeds any gains from hyperparameter optimization
2. Computational Reality: Even modest hyperparameter tuning required substantial computational resources relative to gains achieved
3. Diminishing Returns: The relationship between computational investment and performance improvement is highly non-linear

9.6.5 Methodological Contribution The express tuning methodology provides a replicable template for applied machine learning projects facing computational limitations, demonstrating that: • Strategic parameter selection can achieve comparable results to exhaustive search • Reduced validation scope maintains reliability while improving efficiency • Sample-based training preserves temporal relevance with computational tractability • Pragmatic optimization often proves superior to abandoned comprehensive approaches 9.7 Model Development Summary The systematic algorithm evaluation and enhancement process yielded substantial improvements through a methodical approach:

1. Baseline Establishment: Random Forest selected over XGBoost and Linear Regression based on accuracy and stability

2. Feature Engineering Breakthrough: 13.75% improvement through domain-informed temporal features

3. Optimization Limits: Express hyperparameter tuning demonstrated diminishing returns vs. computational cost

4. Final Performance: MAE of 1.38 passengers on holdout data, representing practical accuracy for operational deployment This progression demonstrates that in applied machine learning for transportation analytics, domain expertise and feature engineering provide greater value than algorithmic sophistication or exhaustive optimization.

5. Discussion This section discusses the key findings derived from the created models and describes the utility these models hold when implemented in the real world. The analysis focuses on methodological insights, business value creation, and practical implications for transportation analytics. 10.1 Methodological Findings and Algorithmic Insights 10.1.1 Feature Engineering Supremacy The most significant finding of this project was that domain-informed feature engineering provided substantially greater performance improvements than algorithm selection. The 13.75 % improvement achieved through advanced temporal features (from 1.60 to 1.38 MAE) far exceeded the performance differences between algorithms, demonstrating where modeling efforts should be concentrated in applied transportation analytics. The dominance of engineered temporal features in the final model - with lag and rolling window features accounting for 67% of predictive power - validates transportation engineering principles about temporal autocorrelation in passenger behavior. 10.1.2 Model Selection Insights: Random Forest vs. XGBoost The comparative analysis revealed fundamental differences in learning strategies between ensemble methods that have important implications for transportation forecasting: Random Forest developed a nuanced, weather-aware approach incorporating six weather variables among its top 15 features, enabling seasonal adaptability through continuous environmental factor consideration. XGBoost relied primarily on categorical rules (is_weekend, is_rush_hour) with minimal weather consideration, creating a more rigid decision framework. This difference explains Random Forest's superior cross-seasonal performance and stability (standard deviation 0.10 vs. 0.12), challenging common assumptions about XGBoost's superiority for tabular data. In transportation contexts

where models must perform consistently across diverse operational conditions, ensemble averaging proved more robust than sequential boosting. 10.1.3 Weather Context Dependencies One of the most significant insights concerned the complex, context-dependent relationship between weather and ridership behavior. Rather than simple weather avoidance patterns, the analysis revealed sophisticated seasonal interactions: • Summer paradox: Warmer temperatures (>20°C) correlated with higher ridership, indicating that increased city activity outweighs alternative transportation choices • Winter precipitation effects: Frozen precipitation created dramatic ridership spikes, while summer rain showed minimal impact • Temporal context importance: The same weather conditions produced different ridership responses depending on seasonal and daily context These findings validate the need for sophisticated feature engineering approaches that capture seasonal interaction effects rather than simple linear weather relationships (Kashfi et al., 2015). 10.1.4 Computational Pragmatism and Resource Management The express hyperparameter tuning approach demonstrated that creative problem-solving can overcome computational limitations while achieving meaningful improvements. Even with substantial constraints (3 folds instead of 6, 800,000 samples instead of full datasets), the methodology identified performance improvements within practical time limits (6.97 minutes vs. projected days for comprehensive search). This experience revealed genuine hardware constraints in applied machine learning while showing how innovative methodological approaches can provide viable alternatives (Falkner et al., 2018). The 95% reduction in computational time while maintaining optimization effectiveness provides a template for practitioners working under similar resource constraints.

10.2 Business Impact and Real-World Utility

10.2.1 Operational Value Creation The final model's MAE of 1.38 passengers represents meaningful operational value for AtB, enabling several practical applications: Proactive Resource Allocation: The prediction accuracy allows anticipation of demand spikes 2-3 hours in advance, enabling dynamic bus deployment decisions that can reduce passenger wait times and improve service reliability. Overcrowding Prevention: By identifying high-demand periods before they occur, the model enables preventive measures including additional service deployment and passenger flow management. Cost Optimization: More efficient resource allocation based on demand predictions can reduce operational costs while maintaining service quality through optimized scheduling and capacity allocation. Service Quality Enhancement: Better demand predictions lead to reduced passenger wait times, improved satisfaction, and more reliable service delivery.

10.2.2 Event-Based Planning Capabilities The model's ability to incorporate event data enables sophisticated planning for special occasions, particularly valuable for AtB's operational strategy: RBK Game Day Planning: The model learns to anticipate predictable ridership patterns with 270% increases at stadium-area stops and 11.18% increases at city center stops, allowing proactive resource deployment. Holiday Period Adjustments: The differentiation between celebration-based events (+99.31% for National Day) and absence-based holidays (-38.94% to -51.64% for Christmas/Easter) enables appropriate service level planning. Academic Calendar Integration: The 37% system-wide impact during student semester periods provides clear guidance for seasonal service adjustments in university cities. 10.2.3 Weather-Responsive Operations The integration of detailed weather data enables service adjustments responding to meteorological conditions: Adverse Weather Response: The model can predict increased demand during challenging weather conditions, supporting decisions about service frequency and capacity allocation. Seasonal Adaptability: Understanding of weather-ridership interactions across different seasons enables more sophisticated operational planning than simple weather-based rules. Dynamic Service Adjustment: Real-time weather integration can support day-of-operation decisions about service levels and resource deployment. 10.3 Methodological Contributions to Transportation Analytics 10.3.1 Time-Series Validation Framework The implementation of expanding-window cross-validation for ridership prediction provides a replicable template for similar transportation forecasting projects. This approach ensures that models are evaluated under conditions that mirror real-world deployment, preventing overly optimistic performance estimates common in transportation ML applications. 10.3.2 Feature Engineering Pipeline The systematic approach to creating lag, rolling, and cyclical features offers a comprehensive methodology for enhancing temporal modeling in transportation applications. The pipeline's success in improving model performance by 13.75% demonstrates the practical value of domain-informed feature creation. 10.3.3 Computational Efficiency Framework The documentation of computational costs and development of the express tuning methodology provides valuable guidance for practitioners facing similar optimization decisions in resource-constrained environments. This contribution is particularly relevant for smaller transit agencies and research organizations with limited computational resources. 10.4 Limitations and Constraints 10.4.1 Data-Related Limitations Manual Event Curation: The labor-intensive process of identifying and categorizing events represents a significant scalability limitation requiring automation for operational deployment. Missing Data Dimensions: The model lacks several potentially valuable data sources including passenger alighting data, real-time traffic conditions, and socio-demographic information. Temporal Scope: Training exclusively on 2024 data may limit generalizability to different years with varying patterns. 10.4.2 Computational Constraints Hardware Limitations: Available computational resources constrained both optimization scope and real-time deployment feasibility, highlighting practical challenges in resource-limited environments. Processing Requirements: The computational intensity of feature engineering raises questions about frequent model retraining feasibility for operational deployment. 10.4.3 Model Interpretability While Random Forest provides feature importance rankings, the complex interactions between temporal, weather, and operational factors remain partially opaque, limiting detailed insights for operational decision-making beyond prediction outputs. 10.5 Industry Context and Comparative Performance [Placeholder for comparison with industry benchmarks - MAE performance relative to similar transit systems globally when available] The achieved MAE of 1.38 passengers per hour prediction represents practical accuracy for operational decision-making in public transportation contexts. This performance enables meaningful business value creation while acknowledging the inherent complexity of predicting human travel behavior. 10.6 Future Work and Development Priorities The systematic approach demonstrated in this

project provides a solid foundation for several key enhancements that would increase operational value and model performance. Operational Implementation: The highest priority for real-world deployment is developing automated event data collection through municipal APIs and structured data sources, replacing the current manual curation process. Additionally, implementing real-time data integration for weather updates, traffic conditions, and service disruptions would enable dynamic prediction adjustments and seamless integration with AtB's existing planning and dispatch systems. Technical Enhancement: Advanced neural network architectures, particularly LSTM networks designed for temporal sequence modeling, could capture more complex patterns than current ensemble methods. Extending the methodology to multi-modal transportation integration and developing uncertainty quantification through confidence intervals would support more sophisticated risk-aware operational decision-making. Research Extensions: Key opportunities include integrating passenger alighting data for comprehensive flow modeling, developing route-specific optimization approaches to capture unique operational characteristics, and extending the forecasting horizon to support annual planning cycles and infrastructure development decisions. Implementation Roadmap: Immediate focus should address automated data collection , followed by real-time integration capabilities, with advanced modeling techniques and multi-modal extensions as longer-term objectives. Each enhancement builds upon the established methodology while maintaining focus on operational feasibility and measurable business value for AtB's transportation network. 11. Conclusion This project successfully developed and evaluated machine learning models for predicting bus ridership in Trondheim, achieving significant prediction improvements through systematic methodology and domain-informed feature engineering. The research culminated in a final model achieving a Mean Absolute Error (MAE) of 1.38 passengers, representing a 13.75% improvement over the baseline Random Forest model. This section evaluates whether the study objectives have been met and reflects on the project journey and lessons learned. 11.1 Assessment of Study Objectives Reflecting on the project's progression and outcomes, I feel confident that the study objectives have been comprehensively met, though the journey proved more complex and extensive than initially anticipated. The primary objective of developing reliable machine learning models for predicting bus ridership has been successfully fulfilled. The final model, achieving an MAE of 1.38 passengers, represents a 13.75% improvement over the baseline and demonstrates practical accuracy suitable for operational decision-making. More importantly, the systematic methodology employed throughout the project ensures that this performance is robust and reliable rather than the result of methodological shortcuts or overfitting. The data infrastructure development objective was met through the successful integration of three disparate data sources into a unified dataset of 18.7 million records. This process proved far more challenging than initially expected, requiring sophisticated temporal alignment procedures and extensive data validation protocols. The experience highlighted that real-world data integration demands significantly more effort than academic examples suggest, but the resulting comprehensive dataset provides a solid foundation for machine learning analysis. The exploratory data analysis objective yielded insights that exceeded expectations in both scope and operational relevance. The discovery of counterintuitive weather relationships, the granular nature of event impacts, and the dominance of academic calendar effects provided crucial guidance for feature engineering strategies. These insights demonstrate that thorough EDA is not merely a preliminary step but a critical component that directly influences model success. The comparative machine learning model development objective was fulfilled through rigorous evaluation of Random Forest, XGBoost, and Linear Regression using consistent 6-fold expanding window cross-validation. The systematic comparison revealed important algorithmic differences that would have been missed through single-split validation, reinforcing the value of robust evaluation methodologies in time-series applications. Perhaps most significantly, the advanced feature engineering objective produced the project's most impactful results. The 13.75% improvement achieved through temporal feature creation demonstrated that domain expertise and creative feature engineering often provide greater value than sophisticated algorithms. This finding has fundamental implications for how machine learning projects should allocate effort and resources. The business impact analysis objective was addressed through comprehensive operational value assessment, translating technical performance metrics into actionable intelligence for AtB. The ability to identify practical applications such as proactive resource allocation and event-based planning demonstrates that the models provide genuine utility beyond academic performance metrics. The documentation and knowledge transfer objective has been fulfilled through this comprehensive report, which provides replicable methodologies, detailed technical implementation guidance, and honest assessment of limitations alongside achievements. The express hyperparameter tuning methodology, in particular, offers practical value for organizations facing similar computational constraints. 11.2 Project Journey and Lessons Learned 11.2.1 Scope Evolution and Challenge Management The project evolved to become significantly more extensive than initially envisioned. What began as a standard machine learning application transformed into a comprehensive transportation analytics study requiring sophisticated data integration, advanced feature engineering, and business impact analysis. This expansion presented considerable challenges in terms of computational requirements, methodological complexity, and time management. The experience of managing this scope expansion proved invaluable, demonstrating the importance of iterative development and adaptive planning in applied machine learning projects. Each phase revealed new complexities that required creative solutions, from computational constraints in hyperparameter tuning to the manual intensive process of event data curation. 11.2.2 Technical Skill Development and Learning Curve The project facilitated an extensive learning curve encompassing multiple dimensions of applied machine learning: Data Engineering Proficiency: Handling 18.7 million records required developing sophisticated memory management techniques, chunked processing strategies, and robust data validation procedures. The experience of working with large-scale datasets provided practical insights into the computational realities of industrial machine learning applications. Feature Engineering Expertise: The systematic approach to creating lag, rolling window, and cyclical features demonstrated the critical importance of domain knowledge in model enhancement. Learning to balance feature complexity with computational tractability proved essential for achieving optimal performance. Cross-Validation Methodology: Implementing expanding window cross-validation for time-series data required deep understanding of temporal dependencies and data leakage prevention. This experience provided foundational knowledge

applicable to any time-series forecasting project. Business Translation Skills: Converting technical performance metrics into operational value propositions required developing the ability to communicate machine learning concepts in business terms, a skill essential for real-world deployment success. 11.2.3 Methodological Insights and Best Practices Several key methodological insights emerged from the project experience: Domain Expertise Supremacy: The 13.75% improvement from feature engineering far exceeded gains from algorithm selection, reinforcing that subject matter expertise often provides greater value than sophisticated algorithms. Computational Pragmatism: The express hyperparameter tuning approach demonstrated that creative resource management can achieve meaningful improvements when comprehensive optimization is computationally prohibitive. Validation Rigor: Consistent application of time-series cross-validation proved essential for reliable performance assessment, revealing stability differences between algorithms that single-split validation would have missed. Iterative Development Value: The systematic progression through baseline comparison, feature engineering, and optimization provided insights at each stage that informed subsequent decisions, yielding better outcomes than less structured approaches. 11.2.4 Unexpected Discoveries and Insights The analysis process yielded several surprising discoveries that enhanced understanding of transportation dynamics: Operational Excellence: AtB's route organization and network structure proved remarkably well-designed, providing a solid foundation for predictive modeling and demonstrating thoughtful transportation planning. Weather Complexity: The counterintuitive relationship between weather and ridership (warmer temperatures increasing rather than decreasing usage) highlighted the importance of considering seasonal context rather than assuming simple linear relationships. Event Impact Granularity: The dramatic difference between system-wide and localized event impacts (2.39% vs. 270% for RBK games) emphasized the critical importance of spatial analysis granularity in transportation forecasting. Academic Calendar Influence: The overwhelming dominance of student semester periods (37% system-wide impact) as a predictive factor exceeded expectations and highlighted the unique characteristics of university cities. 11.3 Research Contributions and Impact This project makes several meaningful contributions to both academic knowledge and practical transportation analytics: Methodological Framework: The systematic approach to transportation demand forecasting, including the express tuning methodology, provides a replicable template for similar projects facing computational constraints. Feature Engineering Pipeline: The comprehensive temporal feature engineering approach offers practical guidance for enhancing time-series models in transportation applications. Business Translation Model: The framework for converting technical performance into operational value demonstrates effective approaches for communicating machine learning benefits to operational stakeholders. Computational Constraint Solutions: The pragmatic approaches developed for handling large-scale datasets and resource-limited optimization provide valuable insights for practitioners working under similar constraints. 11.4 Final Assessment The project has successfully achieved all stated objectives while providing valuable insights that extend beyond the original scope. The final model, with its 13.75% improvement over baseline performance, represents practical accuracy suitable for operational deployment. More importantly, the systematic methodology and comprehensive documentation ensure that the research outcomes can be effectively built upon for future development. The experience demonstrated that applied machine learning projects require not only technical proficiency but also domain understanding, methodological rigor, and creative problem-solving for real-world constraints. The progression from initial baseline (1.60 MAE) to final model (1.38 MAE) represents substantial improvement achieved through systematic application of domain knowledge, advanced feature engineering, and pragmatic optimization strategies. For AtB and similar transit operators, this research provides a foundation for data-driven decision-making that can improve service quality, operational efficiency, and passenger satisfaction. The express tuning methodology offers a practical template for organizations with computational constraints, while the comprehensive feature engineering approach demonstrates the value of domain-specific model enhancement. The project ultimately demonstrates that applied machine learning, when executed with proper methodology, domain awareness, and creative problem-solving, can deliver significant practical value even within resource constraints. The journey from initial concept to operational model has provided invaluable experience in translating theoretical knowledge into practical solutions for real-world transportation challenges.

References Smith, et al. (2020). Sustainable urban mobility: The environmental impact of public transportation systems. Transportation Research Part D: Transport and Environment, 87, 102501.

Movaghar, A., et al. (2025). Climate change mitigation through public transit adoption. Environmental Science & Policy, 145, 78-89.

Göransson, J., & Andersson, H. (2023). Factors that make public transport systems attractive: a review of travel preferences and travel mode choices. European Transport Research Review, 15, 32.

Fatorachian, H. and Kazemi, H. (2025). Sustainable optimization strategies for on-demand transportation systems: Enhancing efficiency and reducing energy use. Sustainable Environment, 11(1). doi:https://doi.org/10.1080/27658511.2025.2464388.

Ceder (2016). Public Transit Planning and Operation. [online] ResearchGate. Available at: https://www.researchgate.net/publication/308747083_Public_Transit_Planning_and_Operation [Accessed 21 Sep. 2025].

Kepaptsoglou, K., & Karlaftis, M. G. (2009). "Transit route network design problem: Review." Journal of Transportation Engineering, 135(8), 491-505.

Luo, Yu-Feng & Bai, Luyi & Chen, Jun & Yan, Xingchen. (2019). Analysis of Space-Time Variation of Passenger Flow and Commuting Characteristics of Residents Using Smart Card Data of Nanjing Metro. Sustainability. 11. 4989.

10.3390/su11184989.

Foell, S., Phithakkitnukoon, S., Veloso, M., Kortuem, G. and Bento, C. (2016). Regularity of Public Transport Usage: A Case Study of Bus Rides in Lisbon, Portugal. Journal of Public Transportation, 19(4), pp.161–177. doi:https://doi.org/10.5038/2375-0901.19.4.10.

Cats, O., & Gkiotsalitis, K. (2021). "Dynamic Public Transport Operations and Control: A Review." Transportation Research Part C: Emerging Technologies, 132, 103375.

Dell'Olio, L., Ibeas, A. and Cecin, P. (01 2011) 'The quality of service desired by public transport users', Transport Policy, 18, pp. 217–227. Available at: https://doi.org/10.1016/j.tranpol.2010.08.005.

Hounsell, N.B., Shrestha, B.P. and Wong, A. (2012). Data management and applications in a world-leading bus fleet. Transportation Research Part C: Emerging Technologies, 22, pp.76–87. doi:https://doi.org/10.1016/j.trc.2011.12.005.

AtBs årsrapport 2024. (2024). AtBs årsrapport 2024. [online] Available at: https://aarsrapport2024.dittatb.no/ [Accessed 21 Sep. 2025]

Ngo, N.S., & Bashar, S. (2024). The impacts of extreme weather events on U.S. Public transit ridership. Transportation Research Part D: Transport and Environment, 137, 104504.

Zhou, M., et al. (2017). Impacts of weather on public transport ridership: Results from mining data from different sources. Transportation Research Part C: Emerging Technologies, 75, 17-29.

Kashfi, S.A., Bunker, J.M., & Yigitcanlar, T. (2015). Understanding the effects of complex seasonality on suburban daily transit ridership. Journal of Transport Geography, 46, 67-80.

Kandiri, A., et al. (2024). Travel time prediction for an intelligent transportation system based on a data-driven feature selection method considering temporal correlation. Transportation Engineering, 18, 100272. Hodson, T.O. (2022). Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. Geosciences Model Development, 15, 5481–5487.

Willmott, C. and Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. Climate Research, 30(1), pp.79–82.

Hodson, T.O. (2022). Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. Geosciences Model Development, 15, 5481–5487.

Chicco, D., Warrens, M.J., & Jurman, G. (2021). The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. PeerJ Computer Science, 7, e623.

Cerqueira, V., Torgo, L., & Mozetič, I. (2020). Evaluating time series forecasting models: an empirical study on performance estimation methods. Machine Learning, 109, 1997–2028.

Tashman, L.J. (2000). Out-of-sample tests of forecasting accuracy: an analysis and review. International Journal of Forecasting, 16(4), pp.437–450. doi:https://doi.org/10.1016/s0169-2070(00)00065-0.

Huang, H., et al. (2022). Bus single-trip time prediction based on ensemble learning. Computational Intelligence and Neuroscience, 2022, 6831167.

AlKhereibi, A.H., et al. (2023). Predictive Machine Learning Algorithms for Metro Ridership Based on Urban Land Use Policies in Support of Transit-Oriented Development. Sustainability, 15(2), 1718.

Zhang, Y., et al. (2022). A XGBoost-Based Lane Change Prediction on Time Series Data Using Feature Engineering for Autopilot Vehicles. IEEE Transactions on Intelligent Transportation Systems, 23(10), 19187-19200.

Falkner, S., Klein, A., & Hutter, F. (2018). BOHB: Robust and Efficient Hyperparameter Optimization at Scale. Proceedings of the 35th International Conference on Machine Learning (ICML), 1436-1445.